

CS 3006
Parallel and Distributed Computing
Lecture 7

Danyal Farhat
FAST School of Computing
NUCES Lahore

Decomposition Techniques

Lecture's Agenda

- **Decomposition Techniques**
- **Recursive Decomposition**
- **Data Decomposition**
- **Exploratory Decomposition**
- **Speculative Decomposition**
- **Hybrid Decomposition**
- **Summary**
- **Additional Resources**

Decomposition Techniques

How to decompose a task into various subtasks?

- While there is **no single recipe** that works for all problems, we present a set of commonly used techniques that apply to broad classes of problems. These include:
 - recursive decomposition
 - data decomposition
 - exploratory decomposition
 - speculative decomposition
- General purpose vs. Special purpose
 - The recursive and data decomposition techniques are relatively **general purpose** as they can be used to decompose a wide variety of problems
 - The speculative and exploratory techniques are more of a **special purpose** nature because they apply to specific classes of problems

Recursive Decomposition

- Generally suited to problems that are solved using the **divide-and-conquer** strategy
- A given problem is first **decomposed** into a set of sub-problems
- These sub-problems are recursively decomposed further until a **desired granularity** is reached
- A natural **concurrency exists** as different sub-problems can be solved concurrently

Finding the Minimum Number

- The problem of finding the minimum number in a given list (or indeed any other associative operation such as sum, AND, etc.) can be fashioned as a divide-and-conquer algorithm.
- We first start with a **simple serial loop** for computing the minimum entry in a given list:
 1. procedure SERIAL_MIN (A, n)
 2. begin
 3. min = A[0];
 4. for i := 1 to n – 1 do
 5. if (A[i] < min) min := A[i];
 6. endfor;
 7. return min;
 8. end SERIAL_MIN

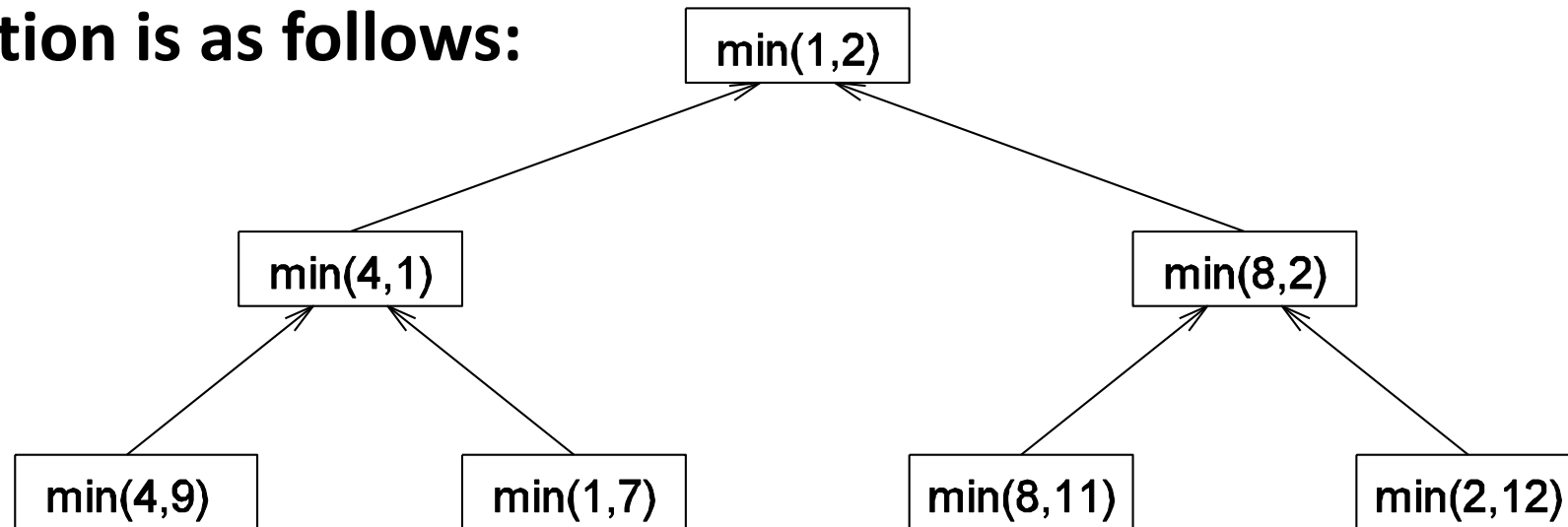
Finding the Minimum Number (Cont.)

```
procedure RECURSIVE_MIN (A, n)
begin
    if (  $n = 1$  ) then
         $min := A[0]$ ;
    else
         $lmin = \text{RECURSIVE\_MIN} ( A, n/2 );$ 
         $rmin = \text{RECURSIVE\_MIN} ( \&(A[n/2]), n - n/2 );$ 

        if ( $lmin < rmin$ ) then
             $min = lmin$ ;
        else
             $min = rmin$ ;
        endelse;
    endelse;
return  $min$ ;
end RECURSIVE_MIN
```

Finding the Minimum Number (Cont.)

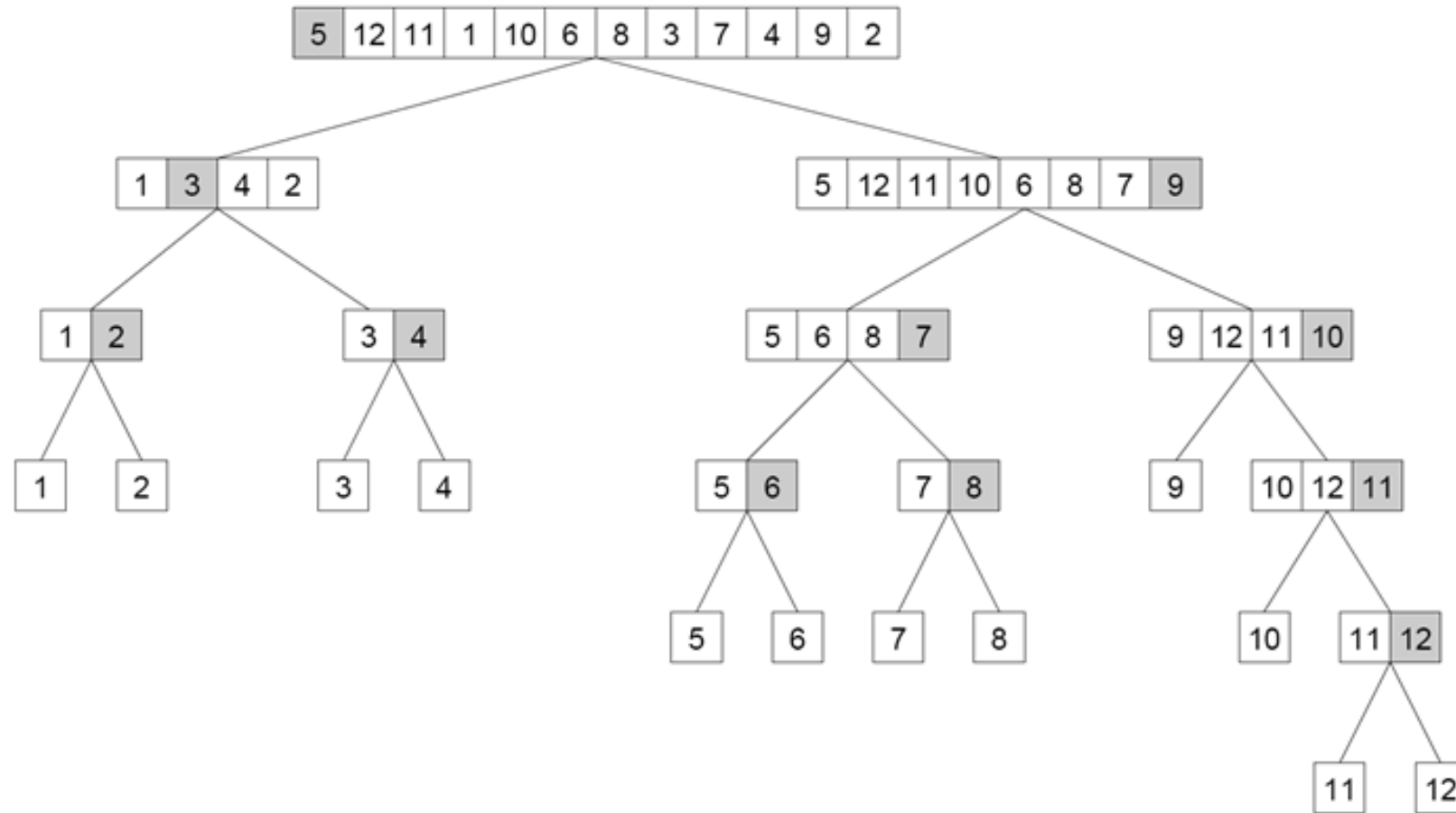
- The code in the previous foil can be decomposed naturally using a recursive decomposition strategy. We illustrate this with the following example of finding the minimum number in the set {4, 9, 1, 7, 8, 11, 2, 12}. The **task dependency graph** associated with this computation is as follows:



Recursive Decomposition Example: Quick Sort

- Once we have selected a **pivot value**, partitioning places all the elements less than the pivot in the left part of the array and all elements greater than the pivot in the right part of the array and the pivot is in the slot between them.
- The pivot element ends up in the position it **retains** in the final sorted order
- After partitioning **no element flops** to the other side of the pivot in the final sorted order
- Thus we can **sort the elements** to the left of the pivot and the right of the pivot independently

Recursive Decomposition Example: Quick Sort



Quicksort Pseudo-code

- **Quicksort(A, low, high)**
 - If (low < high)
 - pivotLocation = Partition(A, low, high)
 - Quicksort(A, low, pivotLocation - 1)
 - Quicksort(A, pivotLocation + 1, high)
- **Partition(A, low, high)**
 - Pivot = A[low]
 - Leftwall = low
 - For (i = low +1 to high)
 - if (A[i] < pivot)
 - leftwall = leftwall + 1
 - Swap (A[i], A[leftwall])
 - Swap(A[low], A[leftwall])

Data Decomposition

- Used for deriving **concurrency** in algorithms that operate on large data structures.
 - Example: Matrix multiplication
- **Identify** the data on which computations are performed.
- **Partition** this data across various tasks.
- This partitioning **induces** a decomposition of the problem.
- Data can be **partitioned** in various ways - this critically impacts performance of a parallel algorithm.
- Often, each element of the output can be **computed** independently of others (but simply as a function of the input).
- A **partition of the output** across tasks decomposes the problem naturally.

Data Decomposition (Cont.)

- Consider the problem of multiplying two $n \times n$ matrices A and B to yield matrix C . The output matrix C can be partitioned into **four tasks** as follows:

$$\begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix} \cdot \begin{pmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{pmatrix} \rightarrow \begin{pmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{pmatrix}$$

$$C_{1,1} = A_{1,1}B_{1,1} + A_{1,2}B_{2,1}$$

$$C_{1,2} = A_{1,1}B_{1,2} + A_{1,2}B_{2,2}$$

$$C_{2,1} = A_{2,1}B_{1,1} + A_{2,2}B_{2,1}$$

$$C_{2,2} = A_{2,1}B_{1,2} + A_{2,2}B_{2,2}$$

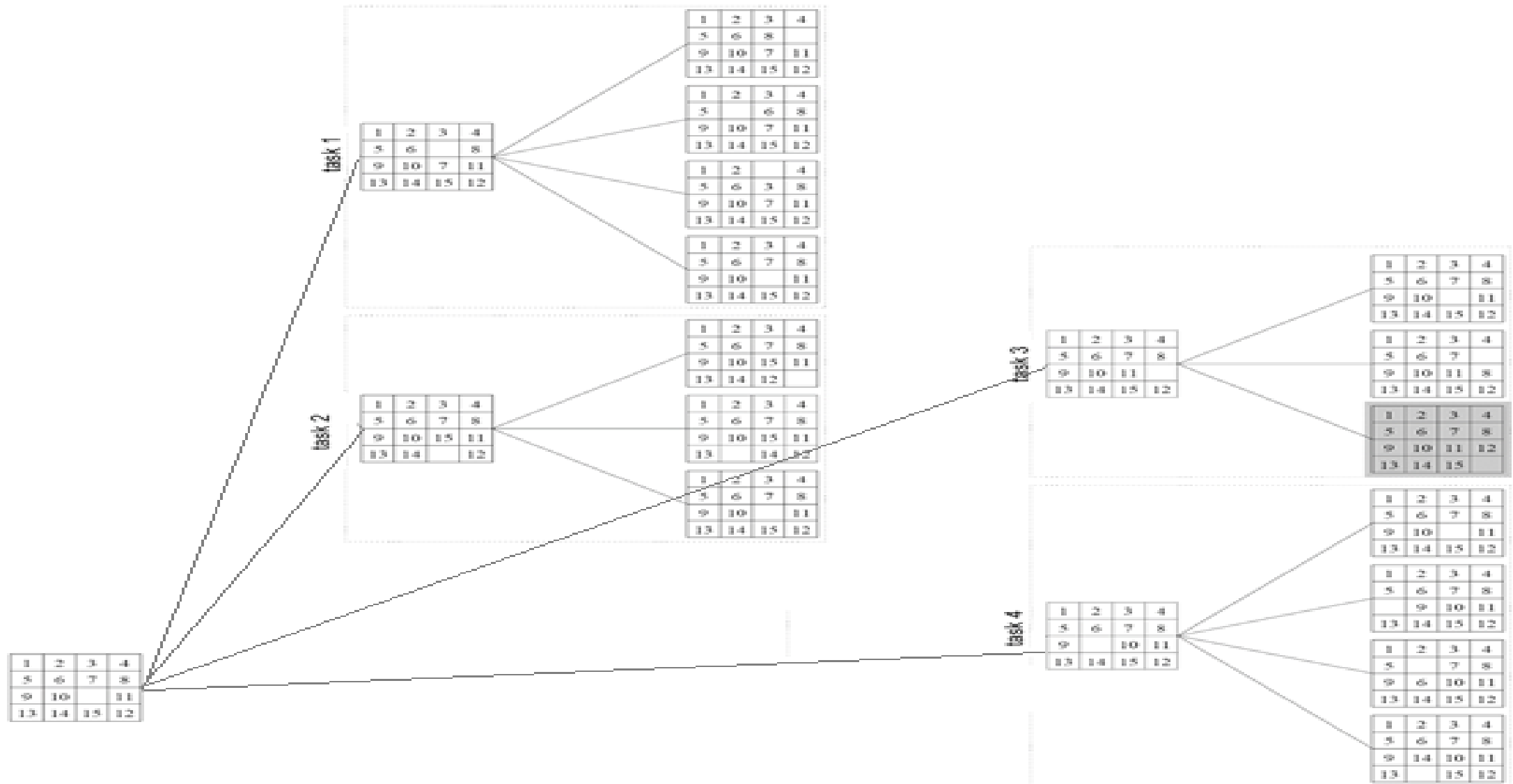
Exploratory Decomposition

- Search through a **predefined space** for solution
- Search space can be **partitioned into tasks** which can be processed independently
- A simple example is **tile puzzle** to transform a given initial state to desired final state

Solution algorithm

- Subsequent configurations are **generated** based on current configuration
- Each configuration is then **explored** as an independent task
- Tasks induced by exploratory decomposition can be **terminated** before finishing as soon as desired solution is found by any one task.

Exploratory Decomposition (Cont.)



Speculative Decomposition

- This decomposition is used when a program may take **one of many possible** computationally significant branches depending on the output of other computations that precede it
- Dependencies between tasks **are not known** a-priori
- For example: a **switch-statement**
- These function/s are **turned into tasks** and carried out before the condition is even evaluated
- As soon as the condition has been evaluated, only the **results of one task** are used, all others, if computed, are thrown away

Speculative Decomposition (Cont.)

- **Steps:**

- Speculate (guess) the **output** of previous stage
- Start **performing computations** in the next stage before even the completion of the previous stage.
- After availability of the output of previous stage, if **speculation was correct** than most of the computation for next step would have already been done.

- **Switch Example:**

Calculate expression for the **switch** condition -> task 0

Case 0: Multiply vector **b** with matrix A -> task 1

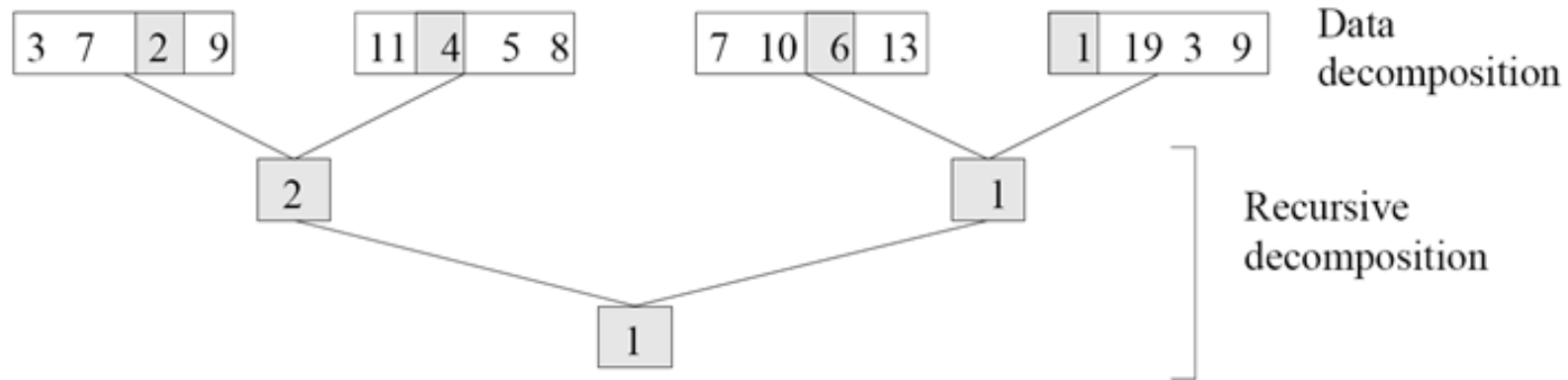
Case 1: Multiply vector **c** with matrix A -> task 2

Case 2: Multiply vector **d** with matrix A -> task 3

display result -> task 4

Hybrid Decomposition

- Decomposition technique are not exclusive
 - We often need to **combine** them together



Summary

- **Decomposition Techniques**
 - No single recipe, multiple techniques, hybrid version can also be used
- **Recursive Decomposition**
 - Divide and conquer strategy
 - Examples: Finding the Minimum Number, Quick sort and many others
- **Data Decomposition**
 - Data of independent tasks is decomposed and mapped on multiple processors for parallel execution
 - Example: Matrix Multiplication

Summary (Cont.)

- **Exploratory Decomposition**
 - Search through a predefined space for solution
 - Example: Tile puzzle
- **Speculative Decomposition**
 - Speculate (guess) the output of previous stage
 - Start performing computations in the next stage before even the completion of the previous stage
 - Example: Switch statement
- **Hybrid Decomposition**
 - Multiple decomposition techniques can be used simultaneously

Additional Resources

- **Introduction to Parallel Computing** by **Ananth Grama** and **Anshul Gupta**

➤ **Chapter 3. Principles of Parallel Algorithm Design**

✓ **Section 3.2: Decomposition Techniques**

Questions?